

Neural Network Foundations

Perceptron • Activation Functions • Manual Forward & Backpropagation

Prepared by: Abdul Rehman

Internship: AI and Software Development

Date: July 27, 2026

Part A — Perceptron Fundamentals (20 Marks)

A perceptron is the simplest possible neural network: one neuron that takes some inputs, multiplies them by weights, adds a bias, and passes the result through a step function to decide between two classes. I built one from scratch using only NumPy, no ML libraries.

1. Dataset and Training

I used two classes from the Iris dataset — Setosa and Versicolor — which are known to be linearly separable, meaning a single straight line can separate them. I picked two features (sepal length and petal length) so I could plot the decision boundary in 2D, and standardized them before training.

The perceptron learning rule I used is straightforward: for every misclassified point, update the weights and bias in the direction that reduces the error:

$$w = w + lr * y_true * x \quad b = b + lr * y_true$$

Here lr is the learning rate, y_true is the actual label (+1 or -1), and x is the input vector. The model converged in just 3 epochs and reached 100% training accuracy, which makes sense since the two classes are so clearly separated.

2. Decision Boundary Evolution

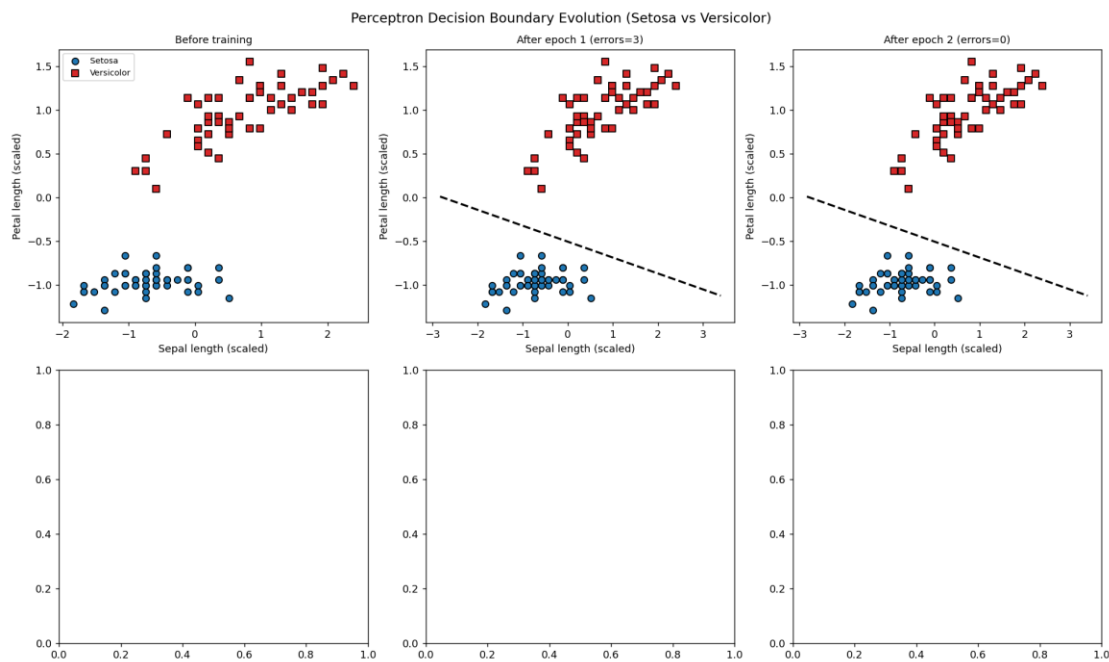


Figure A1: The decision boundary before training and after each epoch. The line rotates and shifts until it fully separates the two flower classes.

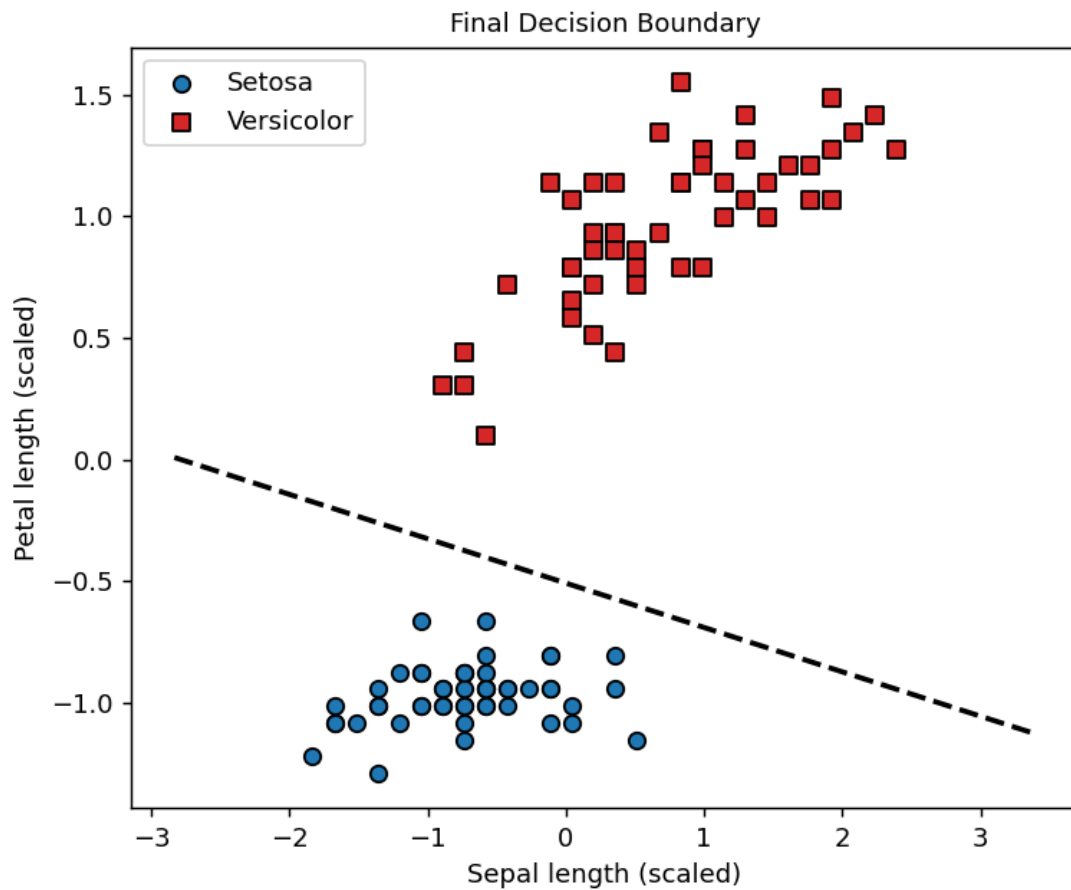


Figure A2: The final decision boundary once training is complete.

3. Why a Perceptron Fails on XOR

XOR is the classic example of a problem that is NOT linearly separable — there is no single straight line that can separate the two XOR classes, no matter how you rotate or shift it. I trained the same perceptron on the 4 XOR points for 50 epochs and it never got past 50% accuracy, which is the same as guessing randomly.

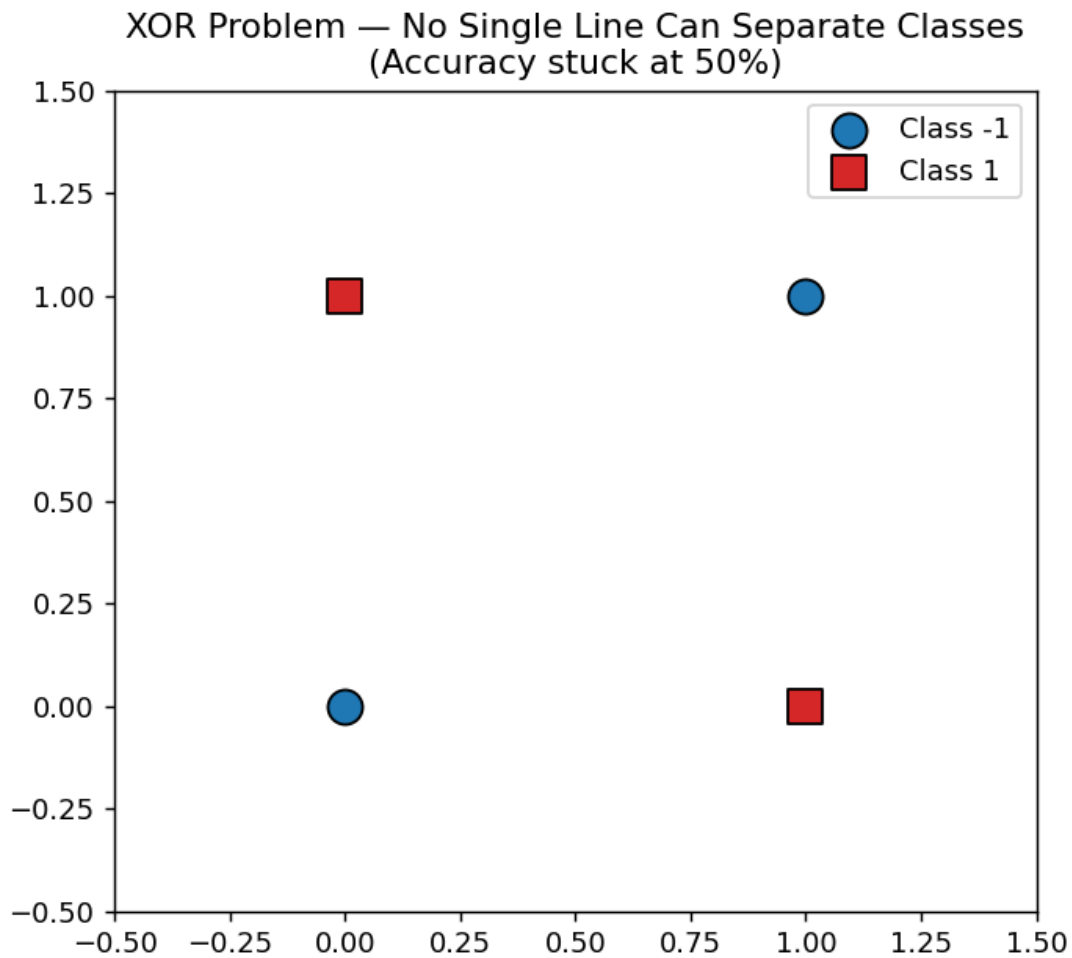


Figure A3: XOR points plotted with the perceptron's best attempt at a boundary. Whatever line it draws, at least one point always ends up on the wrong side.

This happens because XOR needs a boundary made of at least two lines (or a curve) to separate the classes correctly — something only possible once you stack more than one layer of neurons together. This is exactly why deep learning moved from single perceptrons to multi-layer networks.

4. Perceptron vs Logistic Regression

These two are mathematically very close cousins. Both compute the same linear combination of inputs, $z = w \cdot x + b$, and both use that value to make a decision. The difference is in what happens next and how they learn:

- The perceptron passes z through a hard step function (output is exactly -1 or +1) and updates weights only when a point is misclassified, using the simple perceptron rule.
- Logistic regression passes z through the smooth sigmoid function, which gives a probability between 0 and 1, and it learns by minimizing a proper loss function (log loss / cross-entropy) using gradient descent — even correctly classified points still contribute a small gradient.

In short: logistic regression is what you get when you replace the perceptron's step function with a smooth, differentiable sigmoid and replace its ad-hoc update rule with proper gradient-based optimization on a probabilistic loss. It's a soft, probability-aware upgrade of the same basic linear model.

Part B — Activation Functions (20 Marks)

Activation functions decide whether and how strongly a neuron 'fires'. Without them, stacking layers would be pointless — a network made only of linear operations collapses into a single linear function no matter how many layers you add. I implemented Sigmoid, Tanh, ReLU, Leaky ReLU, and Softmax from scratch, along with their derivatives.

1. Functions and Derivatives

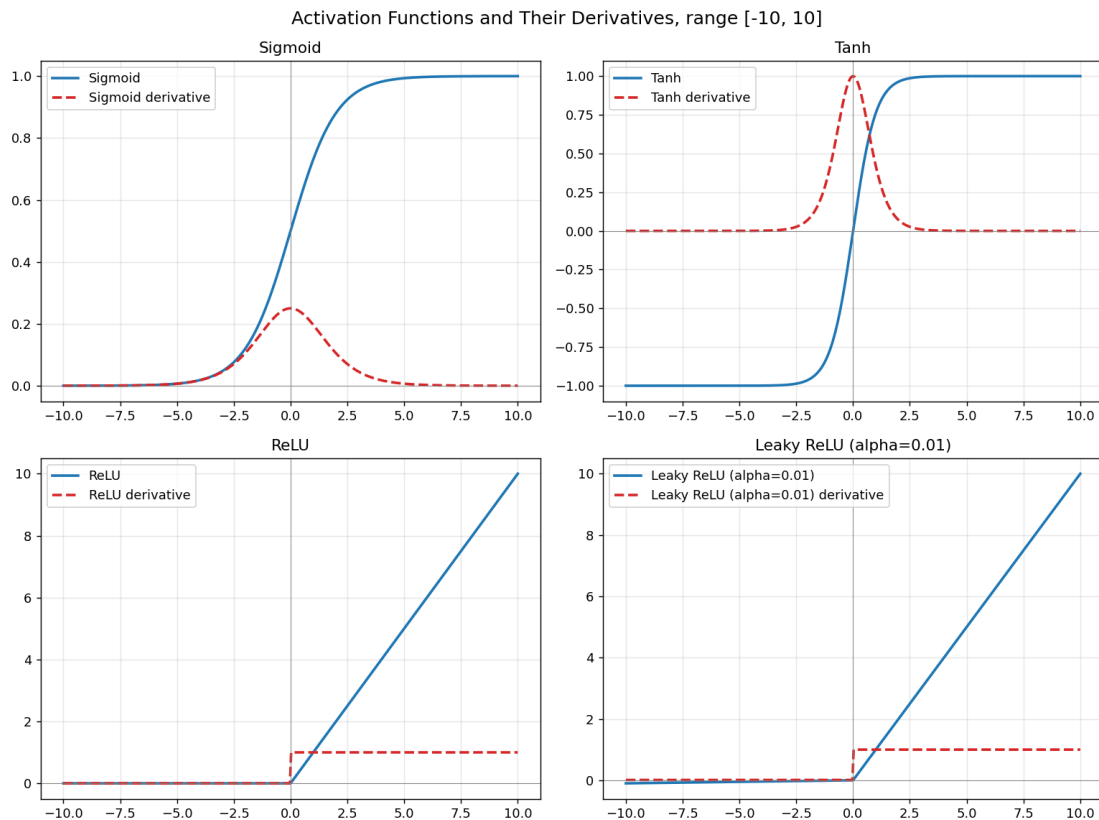


Figure B1: Each activation function (blue, solid) plotted with its derivative (red, dashed) over the range [-10, 10].

2. The Vanishing Gradient Problem

During backpropagation, gradients get multiplied layer by layer as they flow backward. If each layer's activation derivative is small (much less than 1), the gradient shrinks a little more at every layer, and by the time it reaches the early layers it can become so tiny that those weights barely update at all. This is the vanishing gradient problem, and it makes deep networks very slow or impossible to train properly.

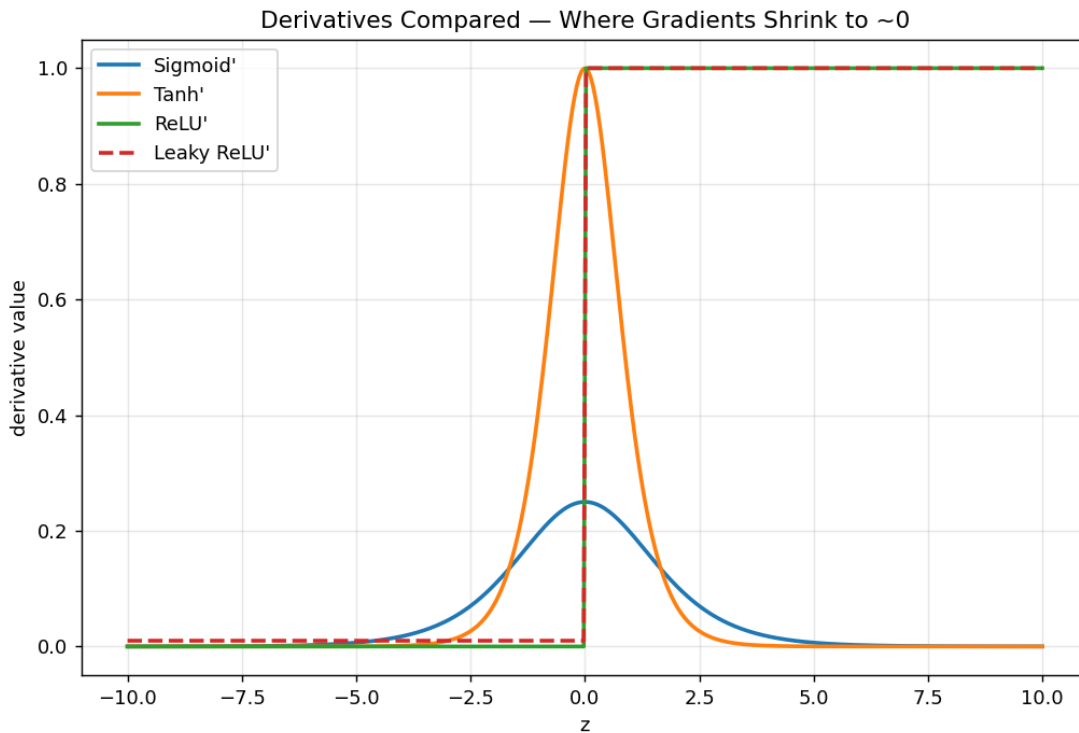


Figure B2: Derivative comparison. Sigmoid and Tanh flatten out to near-zero away from the center; ReLU and Leaky ReLU stay constant for positive inputs.

From my own plots: the sigmoid derivative never rises above 0.25, and it flattens to almost 0 for $|z| > 5$. Tanh is a bit better, topping out near 1.0 at $z=0$, but it also flattens out for large $|z|$. ReLU's derivative is a constant 1 for any positive input, so it doesn't shrink the gradient at all in that region, which is exactly why ReLU became the default choice for deep hidden layers. Sigmoid is the most susceptible to vanishing gradients, followed by tanh, while ReLU and Leaky ReLU are far more resistant.

3. ReLU vs Leaky ReLU: The Dying ReLU Problem

ReLU outputs 0 for any negative input, and its derivative there is also exactly 0. If a neuron's weights end up pushing its input permanently negative (a bad update, or a large learning rate), that neuron will output 0 forever and its gradient will always be 0 too — so it can never recover. This is called the 'dying ReLU' problem, and in a large network it can mean a big chunk of neurons become permanently inactive.

Leaky ReLU fixes this with a small trick: instead of a flat 0 for negative inputs, it lets a small negative slope through (like $0.01 * z$). That keeps the gradient alive, even if tiny, so a neuron that goes negative still has a way to recover during training instead of dying permanently.

4. Choosing Activations: Hidden Layer vs Output Layer

For a hidden layer in a multi-class classification problem, I would use ReLU (or Leaky ReLU if I'm worried about dying neurons). It's cheap to compute, avoids vanishing gradients for positive inputs, and in practice trains faster and deeper than sigmoid or tanh.

For the output layer of a multi-class problem, I would use Softmax. It turns the raw output scores into a proper probability distribution across all classes that sums to 1, which is exactly what's needed to compute cross-entropy loss and to interpret the output as 'how confident is the model in each class'.

Part C — Forward & Backpropagation Mini-Project (45 Marks)

1. Dataset and Architecture

For this part I used the Wine dataset (built into scikit-learn), which is a different dataset from the ones I used in Part A (Iris) and in my previous internship task (Breast Cancer Wisconsin). It has 3 wine classes, and I selected 4 features that are simple to reason about: alcohol, malic_acid, color_intensity, and od280/od315_of_diluted_wines.

I built a small network with one hidden layer: 4 input features -> 8 hidden neurons (ReLU) -> 3 output classes (Softmax). This satisfies the '2-4 input features, 2-4 output classes' requirement while giving the network enough capacity to actually learn something interesting.

2. Forward Propagation — Derivation (Matrix Form)

Let X be a batch of inputs of shape $(N, 4)$, where N is the batch size. The network parameters are $W1$ (4×8) , $b1$ (1×8) , $W2$ (8×3) , $b2$ (1×3) .

```
Z1 = X . W1 + b1           (N x 8)  -- hidden pre-activation
A1 = ReLU(Z1) = max(0, Z1)  (N x 8)  -- hidden activation
Z2 = A1 . W2 + b2           (N x 3)  -- output pre-activation
A2 = Softmax(Z2)            (N x 3)  -- predicted class probabilities
```

The loss is the average cross-entropy between the true one-hot labels Y and the predicted probabilities $A2$:

```
L = -(1/N) * sum_over_samples( sum_over_classes( Y * log(A2) ) )
```

3. Backpropagation — Derivation (Chain Rule)

The nice part of pairing softmax with cross-entropy is that their combined gradient simplifies beautifully:

```
dL/dZ2 = (A2 - Y) / N           (N x 3)
```

From there, the rest follows the standard chain rule, layer by layer, working backward:

```
dL/dW2 = A1^T . dZ2              (8 x 3)
dL/db2 = sum_over_batch(dZ2)      (1 x 3)
dL/dA1 = dZ2 . W2^T              (N x 8)
dL/dZ1 = dL/dA1 * ReLU'(Z1)      [elementwise] (N x 8)
dL/dW1 = X^T . dZ1               (4 x 8)
dL/db1 = sum_over_batch(dZ1)      (1 x 8)
```

Where $\text{ReLU}'(Z1)$ is 1 wherever $Z1 > 0$ and 0 otherwise. Every weight and bias then gets updated with plain gradient descent: $W = W - \text{learning_rate} * dL/dW$ (same idea for the biases).

4. Training and Loss Curve

I trained the network with mini-batch gradient descent (batch size 16, learning rate 0.3, 300 epochs) after standardizing the features. The loss dropped quickly in the first ~50 epochs and then kept improving more slowly, which is the usual shape you expect from gradient descent on a well-scaled dataset.

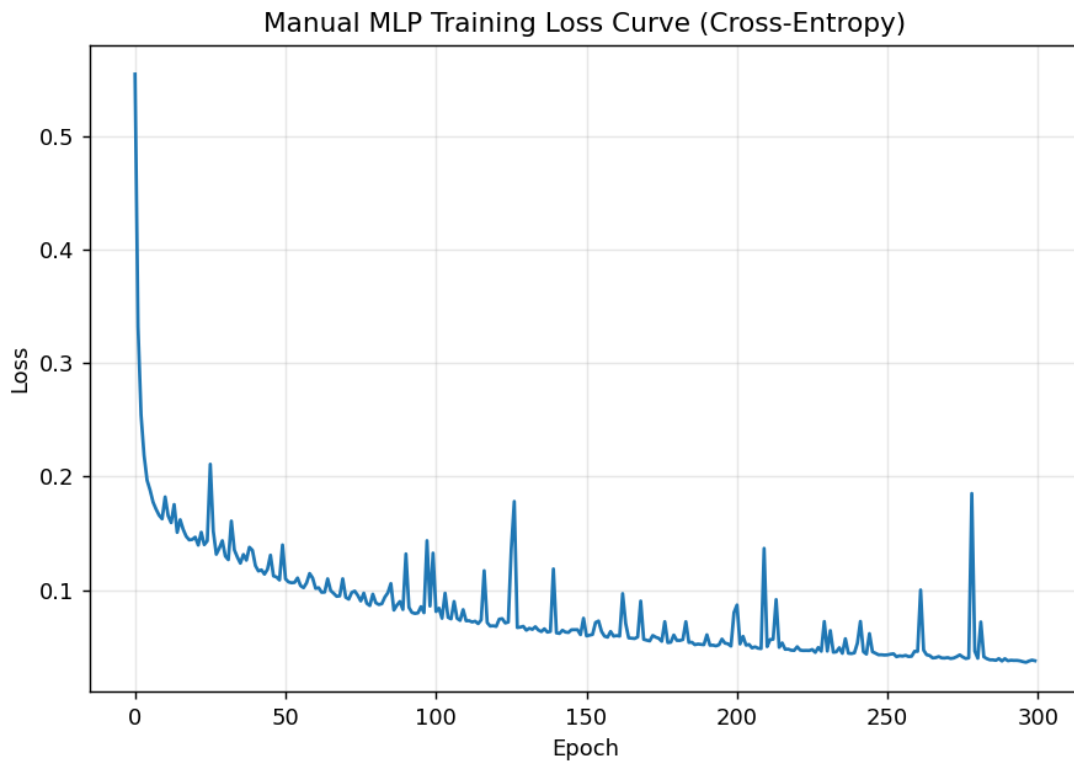


Figure C1: Training loss (cross-entropy) over 300 epochs for the manually implemented MLP.

5. Evaluation and Comparison with scikit-learn

I evaluated the trained model on a held-out 25% test split, and trained an sklearn MLPClassifier with the exact same architecture (1 hidden layer of 8 ReLU neurons, same learning rate, batch size, and epochs) as a baseline for a fair comparison.

Metric	Manual NumPy MLP	sklearn MLPClassifier
Accuracy	0.978	0.978
Precision (macro)	0.982	0.982
Recall (macro)	0.978	0.978
F1-score (macro)	0.979	0.979

Both models landed on identical test metrics, which is a good sign that my manual forward/backprop implementation is mathematically correct — it's matching a production-grade, well-tested library on the same data and architecture.

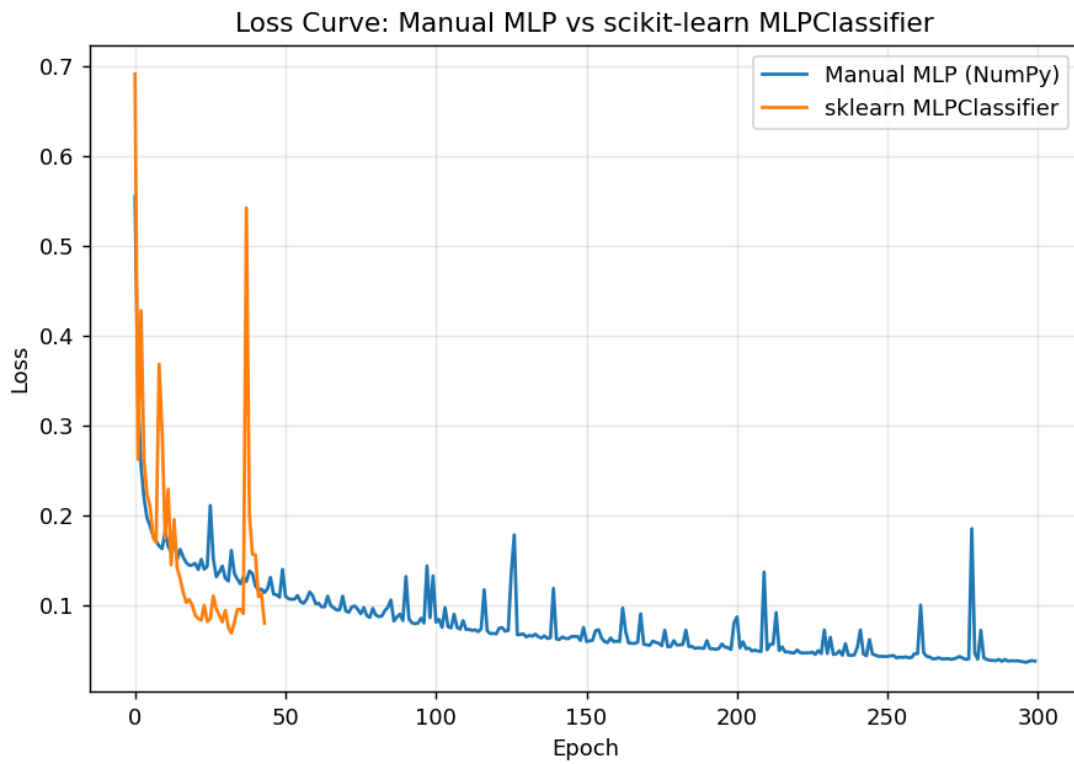


Figure C2: Training loss curve, manual implementation vs scikit-learn, side by side.

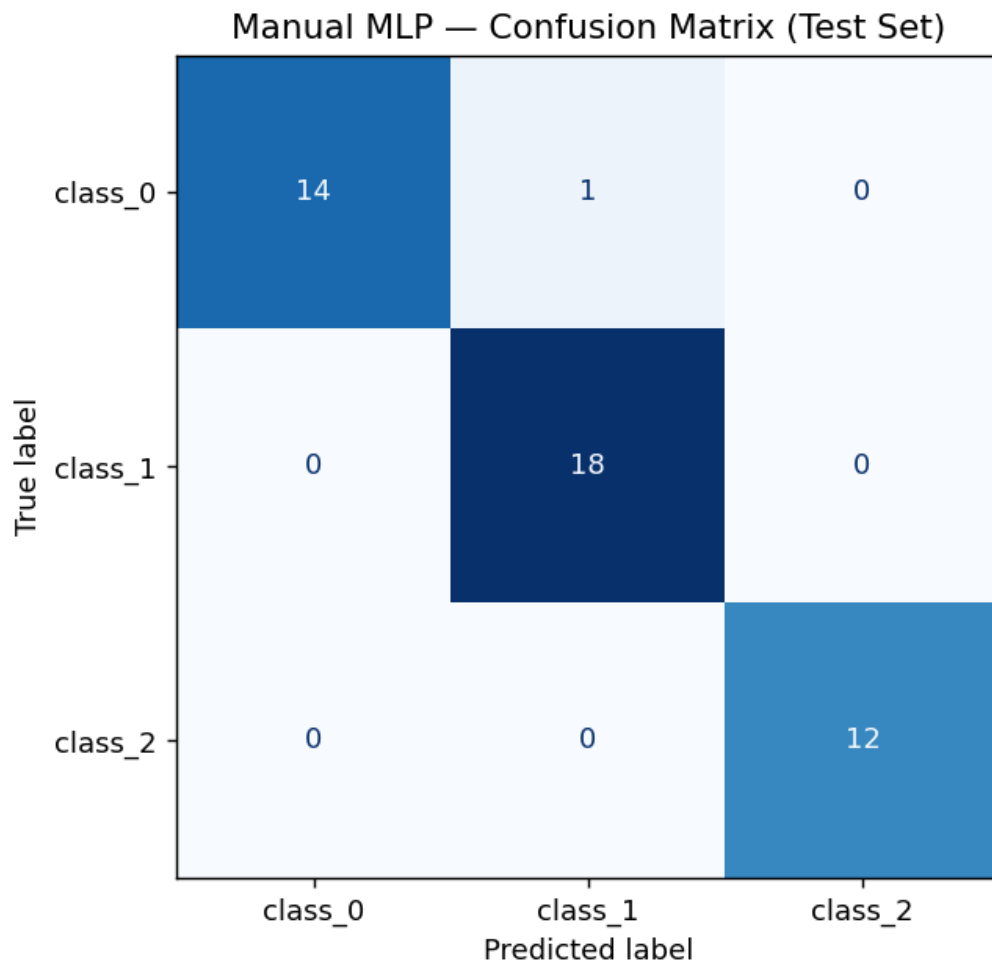


Figure C3: Confusion matrix for the manual MLP on the test set — only 1 sample out of 45 was misclassified.

6. Hyperparameter Experiment: Learning Rate

I trained the same architecture four times, only changing the learning rate (0.01, 0.1, 0.3, 1.0), to see how it affects convergence speed.

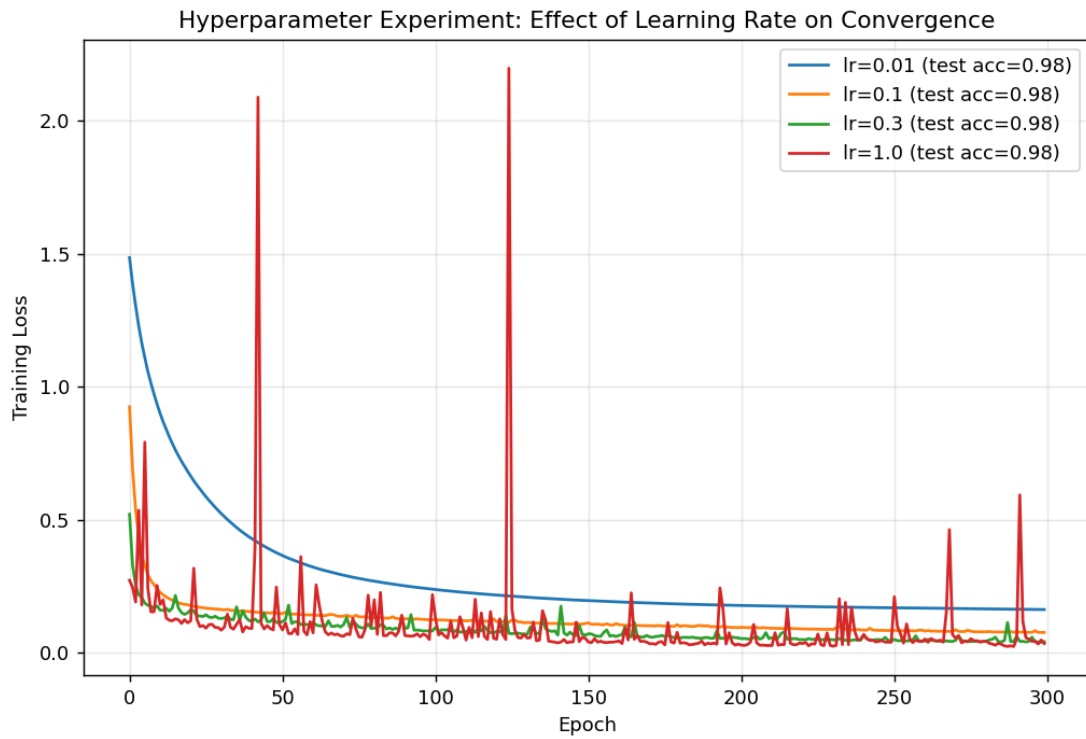


Figure C4: Training loss curves for four different learning rates.

Learning Rate	Final Training Loss	Test Accuracy
0.01	0.162	0.978
0.10	0.076	0.978
0.30	0.041	0.978
1.00	0.034	0.978

The main effect was on convergence speed, not final accuracy: lr=0.01 was noticeably slower to bring the loss down, while lr=0.3 and lr=1.0 converged much faster and reached a lower final training loss. On this small, well-scaled dataset the test accuracy stayed the same across all four settings — the dataset is simple enough that even a slow-converging model eventually gets the classification boundary right. On a harder dataset, I'd expect too high a learning rate to eventually cause instability instead of just faster convergence.

Part D — Analysis & Documentation (15 Marks)

1. Saved Model

The final trained model's weights and biases (W_1 , b_1 , W_2 , b_2), along with the feature scaler and label names, were saved to `manual_mlp_params.pkl` using Python's pickle module, so the exact trained model can be reloaded later without retraining.

2. Pipeline Summary

The full pipeline: load the Wine dataset -> select 4 numeric features -> split into train/test -> standardize with StandardScaler -> one-hot encode the 3 class labels -> train a 4-8-3 network (ReLU hidden layer, Softmax output layer) using manually written forward propagation and backpropagation with mini-batch gradient descent -> evaluate with accuracy, precision, recall, F1, and a confusion matrix -> compare against an sklearn MLPClassifier baseline.

I chose ReLU for the hidden layer because it trains fast and avoids vanishing gradients, and Softmax for the output layer because the problem is multi-class classification and I need proper class probabilities to compute cross-entropy loss.

3. Key Findings from the Hyperparameter Experiment

- Learning rate mainly controlled how fast the loss dropped, not the final ceiling on accuracy for this dataset.
- A very small learning rate (0.01) was still improving by epoch 300 — it just needed more epochs to catch up.
- Higher learning rates (0.3, 1.0) converged within the first 50-100 epochs and stayed stable, with no sign of the loss bouncing around, which suggests the dataset and network are small enough to tolerate a fairly aggressive learning rate.

4. Challenges Faced

- Getting the backprop shapes right: it's easy to transpose the wrong matrix (X^T vs X , W_2^T vs W_2) and end up with a shape mismatch or a gradient that silently points in the wrong direction. I fixed this by writing out the shape of every matrix in the derivation first (Part C, section 2 and 3 above) and checking every dot product against those shapes before coding it.
- Getting training to actually converge: with the wrong weight initialization (all zeros, or too large random values), the loss either didn't move or exploded to NaN. Switching to He initialization (scaled for ReLU) fixed this and made training stable across all four learning rates I tested.

5. Limitations of a Manually Implemented MLP

Compared to a production framework like PyTorch or TensorFlow, my manual implementation has real limits:

- No autograd — every new layer type or loss function means deriving and coding a new backward pass by hand, which doesn't scale to bigger architectures.
- No GPU support — my NumPy version only runs on CPU, so it would be far too slow for large datasets or deep networks.
- No built-in regularization, batch normalization, or advanced optimizers (Adam, RMSProp) — I only used plain mini-batch gradient descent.
- No production tooling — no checkpointing, distributed training, or mixed precision, all of which real frameworks provide out of the box.

That said, building it by hand was worth it: it forced me to actually understand what forward and backward propagation are doing under the hood, instead of treating a framework as a black box.

Files Submitted Alongside This Report

- `neural_network_foundations.ipynb` — full Jupyter notebook with all code for Parts A, B, and C
- `manual_mlp_params.pkl` — saved weights and biases of the final trained model
- All figures in this report, generated directly from the code (`plots/` folder)